Assignment 1: Shuttle Dispatch

Value and Policy Iteration

GitHub username: FailedMath3U **Repository:** MLDS
`https://github.com/FailedMath3U/MLDS`

1 MDP and modeling assumptions

At decision period t, the state $s_t \in \{0, \dots, 200\}$ is the number waiting before dispatch. The action $a_t \in \{0, 1\}$ means wait or dispatch one shuttle. Its capacity is 15. Let N_t be the subsequent arrivals, independently uniform on $\{1, 2, 3, 4, 5\}$. Define the passengers remaining after service as $q(s, a) = \max\{s - 15a, 0\}$. The cost and next state are

$$c(s, a) = 100a + 2q(s, a), \qquad s_{t+1} = \min\{q(s_t, a_t) + N_t, 200\}.$$

Thus dispatch precedes holding cost, and new arrivals enter afterward. Queue length is sufficient for the Markov property under independent arrivals and unrestricted shuttle availability.

The transition probability $P(s' \mid s, a)$ is the probability of next queue s' given state s and action a:

$$P(s' \mid s, a) = \frac{1}{5} \sum_{k=1}^{5} \mathbb{I}\{s' = \min\{q(s, a) + k, 200\}\}.$$

$\mathbb{I}\{\cdot\}$ equals one when its condition holds and zero otherwise. Arrival outcomes that reach the same capped state have their probabilities added.

Additional assumptions. At most one shuttle departs each period; there is no fleet-size or travel-time constraint. Overflow above 200 is discarded without penalty. New arrivals first incur holding cost at the next decision. The finite-horizon problem has 500 decisions ($t = 0, \dots, 499$) and no terminal cost. Ties choose waiting.

Objective and notation. With discount $\gamma = 0.95$, minimize expected discounted cost. $J_t(s)$ is the minimum remaining cost from state s at time t, discounting relative to that time:

$$J_t(s) = \min_{\pi} \mathbb{E}_{\pi}\left[\sum_{u=t}^{499} \gamma^{u-t} c(s_u, a_u) \,\middle|\, s_t = s\right].$$

Here u counts future periods, π is a rule for choosing actions, and $\mathbb{E}_{\pi}$ averages arrival randomness under that rule. For an infinite horizon, $J^*(s)$ denotes the optimal cost with no deadline. Plots show cost values: lower is better. If rewards are defined as $-c$, the corresponding reward values are $-J$.

Run the code. From `assignment1/mlds_assignment1_shuttle/`:

```
python -m pip install -r requirements.txt
python shuttle.py
jupyter lab shuttle_plots.ipynb
```

The solver runs all three methods and 14 validation experiments. Evidence is in `results/summary.json`; `state_results.csv` lists all states, and `shuttle_results.npz` saves every finite-horizon value/policy. The notebook regenerates plots. Section 5 explains each validation experiment.

2 Part (a): enumeration, $T = 500$

We enumerate every state, action and arrival outcome by backward induction. Set $J_{500}(s) = 0$; then for $t = 499, \ldots, 0$ compute

$$Q_t(s,a) = c(s,a) + \frac{\gamma}{5} \sum_{k=1}^{5} J_{t+1}(\min\{q(s,a) + k, 200\}), \qquad J_t(s) = \min_{a \in \{0,1\}} Q_t(s,a).$$

$Q_t(s,a)$ is the cost of action a now followed by optimal decisions. The minimizing action defines the time-dependent policy $\pi_t(s)$. All 501 value vectors and 500 policy vectors are saved. This reuses continuation values rather than listing exponentially many complete histories; an independently enumerated small tree validates the recursion (Section 5).

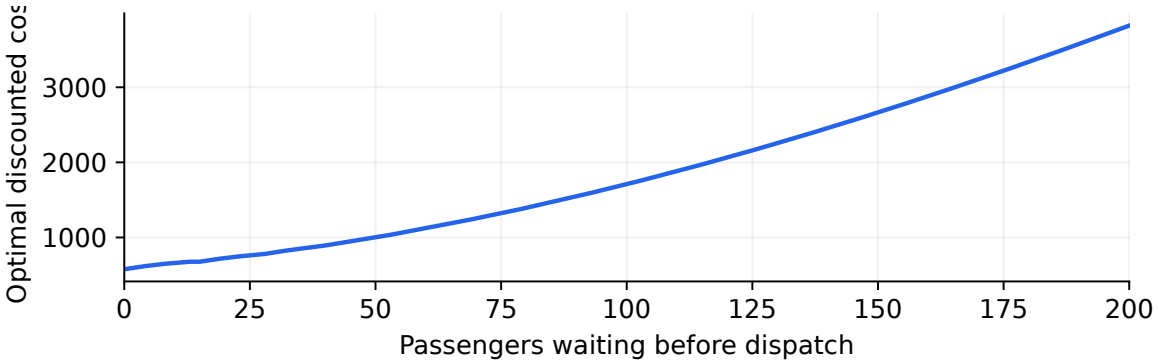


Figure 1: Part (a): optimal discounted cost at time zero, $J_0(s)$, for 500 decisions and zero terminal cost.

3 Part (b): infinite-horizon value iteration

Starting from $J^{(0)}(s) = 0$, update

$$J^{(n+1)}(s) = \min_{a \in \{0,1\}} \left[c(s,a) + \frac{\gamma}{5} \sum_{k=1}^{5} J^{(n)}(\min\{q(s,a) + k, 200\}) \right].$$

The index n counts iterations, not decision periods. Let $\mathcal{T}J$ denote this entire update applied to a candidate vector J. Stop when the Bellman residual $\delta(J) = \max_s |(\mathcal{T}J)(s) - J(s)|$ is at most 10^{-10}. This took **518 updates**, with final residual 9.913×10^{-11}.

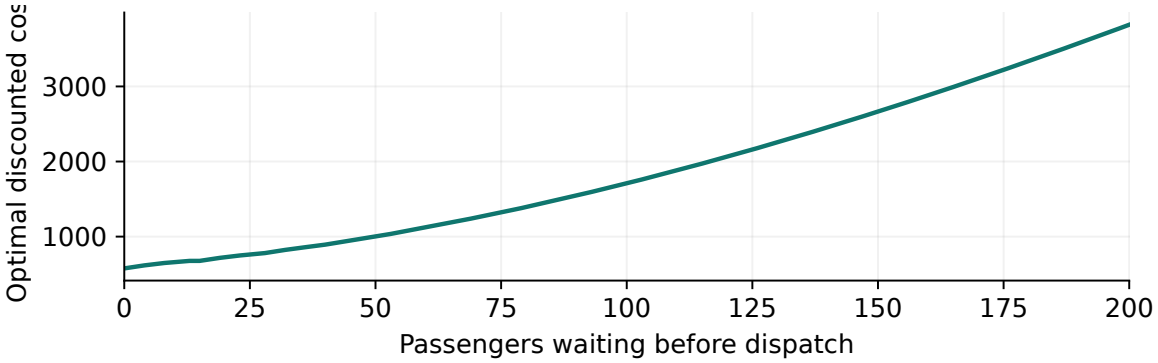


Figure 2: Part (b): infinite-horizon optimal cost $J^*(s)$ from value iteration. Its near agreement with Part (a) is tested using a discounted-tail bound.

4 Part (c): policy iteration and optimal policy

Start from always waiting. For a stationary policy π, form its transition matrix P_π and cost vector c_π, where $P_\pi(s, s') = P(s' \mid s, \pi(s))$ and $c_\pi(s) = c(s, \pi(s))$. Its value vector J_π satisfies

$$(I - \gamma P_\pi) J_\pi = c_\pi.$$

I is the identity matrix. Solve this linear system, then improve each state's action by comparing current cost plus discounted continuation under J_π. Repeat until no action changes. Policy iteration took **six evaluation/improvement rounds**, including the final stability check.

The optimal stationary policy is to **wait at queues 0–12 and dispatch at queues 13–200**. No threshold restriction was imposed on the search.

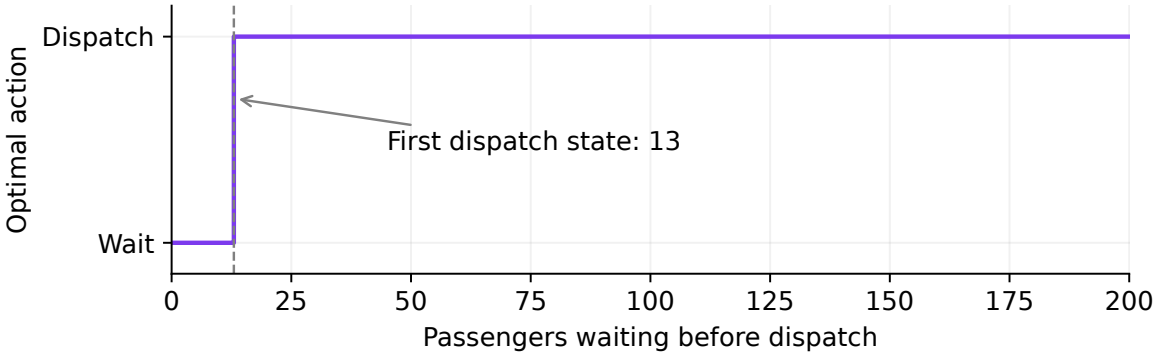


Figure 3: Part (c): optimal stationary policy. Dispatch begins at 13 passengers, before the shuttle is full.

Why dispatch before 15? The action costs, including optimal future costs, switch order between 12 and 13:

Queue	Wait cost	Dispatch cost	Decision
12	672.892	676.670	Wait
13	680.770	676.670	Dispatch

Waiting for a full shuttle is feasible but more expensive: exact policy evaluation gives cost 597.356 from an empty queue, versus 576.670 for the optimal policy. Representative optimal costs are $J^*(15) = 676.670168$ and $J^*(200) = 3821.586477$.

Edge cases and their consequences

Empty and underfilled shuttles. The maximum with zero prevents negative queues. Dispatching at 0 costs 100 extra without changing the next-state distribution, so it is dominated. Dispatch at 15 clears the queue; at 16 it leaves one. Also, $J^*(15) - J^*(0) = 100$, since optimal dispatch at 15 and waiting at 0 leave identical queues before arrivals.

Station capacity. At 200, waiting leads to 200 with probability one; dispatch leads to 186–190 with probability 1/5 each. Repeated capped outcomes must be pooled, not treated as distinct next states with lost probability.

The final decision. Waiting is optimal everywhere at $t = 499$: dispatch costs 100 but saves at most $2 \times 15 = 30$ immediately. With no terminal penalty, there is no later benefit. The saved final-period policy and terminal value were checked against this prediction; the infinite-horizon rule must not be used at every finite-horizon time.

5 Quantitative validation and execution

All **14 saved validation experiments passed**. Agreement alone is insufficient because solvers share a model; the tests also use analytic modifications, independent tree enumeration and independently coded simulation.

Optimality and solver agreement. Value and policy iteration choose identical actions in all 201 states; their maximum value difference is 1.964×10^{-9}. Policy iteration's Bellman residual is 9.095×10^{-13}. Since discounting makes the update contractive,

$$\max_s |J(s) - J^*(s)| \leq \frac{\delta(J)}{1-\gamma},$$

so the residual implies a value-error bound of 1.819×10^{-11}, subject to rounding. The smallest action-cost gap is 3.778, far larger than numerical error. Finite-horizon time-zero values differ from infinite-horizon values by at most 4.942×10^{-9}, below the omitted-tail bound $470\gamma^{500}/(1-\gamma) = 6.838 \times 10^{-8}$; 470 bounds any stage cost.

Modified problems with known answers. Each prediction below matched all states:

Modification	Predicted and observed result
Holding cost set to zero	Always wait; optimal cost zero.
Zero capacity and no arrivals	Always wait; $J(s) = 2s/(1-\gamma)$; zero analytic error.
Free dispatch, capacity 200, no arrivals	Dispatch at nonempty queues; cost zero.
Discount set to zero	Always wait; $J(s) = 2s$.

An exhaustive scalar decision/arrival tree with seven states and four periods matched backward induction exactly. This checks the recursion without reusing its vectorized transition arrays.

Independent simulation. For each initial queue, simulate 20,000 trajectories of 300 decisions with seed 1668, coding costs/transitions directly rather than using the solver's array builder:

Initial queue	Simulated mean	Exact policy cost	Standard error
0	576.671469	576.670168	0.219213
15	676.814669	676.670168	0.221520
200	3821.605190	3821.586477	0.532280

Standard error is trajectory-cost sample SD divided by $\sqrt{20{,}000}$. Every discrepancy is below one SE; the automated tolerance is four SE plus a 0.001951 omitted-tail bound. Simulation checks policy evaluation; Bellman optimality supplies the optimality evidence.

Rebuild the report. Run `python make_report_figures.py`, then compile `SHUTTLE_REPORT.tex`. The builder audits saved arrays and reproduces the full-shuttle comparison without rerunning or overwriting the original solvers.

With three additional months: test cost, capacity, arrival and queue-cap sensitivity; compare overflow penalties and terminal costs. Independently solve the MDP by linear programming and examine near-tied actions at higher precision. Finally, test the policy under correlated demand and explicit shuttle availability/travel times, adding those variables to the state where necessary. These distinguish numerical verification from robustness to modeling assumptions.